

Module 4 Architecture Data and AI Integration

Overview and study route

Software Engineering in the Era of AI | Master's year one | Electrical and Information Engineering | Weeks 8 to 10 | 2026 to 2027

Architecture organises responsibilities and makes trade-offs explicit. An AI feature adds a variable external dependency to a software system; it does not remove the need for data contracts, access control, failure handling or a deterministic baseline. This module develops a small telemetry service whose numerical results remain independently checkable while an optional AI component explains approved supporting information.

Week 8 focuses on boundaries and design decisions. Week 9 develops relational data and retrieval. Week 10 integrates an AI adapter with response validation, evaluation and bounded tool use. The core implementation remains small. Embedding databases, distributed services and a full MCP server are extensions.

Learning outcomes

- Defend a modular architecture using concrete quality requirements and alternatives.
- Specify interfaces, errors and data ownership across component boundaries.
- Model telemetry and provenance in a relational database and use parameterised SQL.
- Distinguish deterministic lookup, retrieval-augmented generation and model training.
- Validate structured model responses separately from their factual content.
- Design an AI adapter with explicit budgets, failure policies and version records.
- Explain tool calling and MCP without treating a protocol as an authorisation system.

This module supports LO1, LO2, LO4, LO5 and LO6.

1 Begin with quality scenarios

A statement such as "the system must be fast and scalable" is too vague to guide a design. A quality scenario identifies a stimulus, operating conditions, required response and measurable target. For the classroom service: when a user submits a valid batch of up to 1000 interval records on the specified lab machine, the deterministic energy result should be available within a locally measured target. Record the machine and target before the performance test.

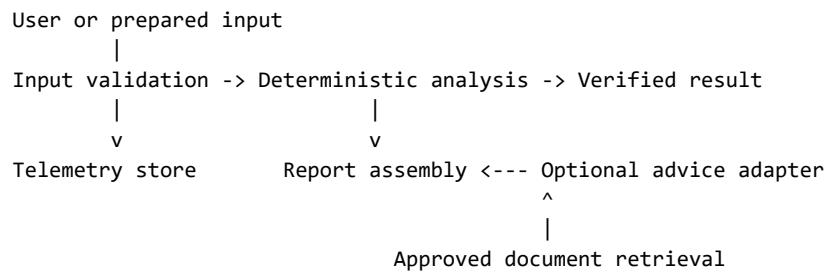
Separate targets for deterministic calculation and AI advice. A model call may take longer, fail or be unavailable. Users should still be able to obtain the verified numerical result if that is the core requirement. The interface should identify whether advice is unavailable rather than fabricate a successful explanation.

Quality attributes can conflict. More logging may help diagnosis but expose sensitive content. More retries may improve completion for transient failures but increase delay and cost. Caching may reduce latency but return stale or unauthorised data. A design decision should name the trade-off and evidence needed to revisit it.

2 A modular monolith as the baseline

A modular monolith runs as one application while keeping internal responsibilities separate. It is often sufficient for a small student project because it reduces deployment and network complexity while preserving clear interfaces. Microservices separate components into independently deployed services; that can support organisational or scaling needs but adds network failures, version coordination and operational work.

The proposed components are an input adapter, validation, deterministic analysis, storage, retrieval, optional AI advice and reporting. The dependency direction should protect the domain calculation from provider-specific details. The energy function does not import an AI SDK.



The diagram shows responsibility and data flow, not a requirement for separate servers. The report assembler must distinguish verified numerical output from generated explanation. A generated sentence cannot override a calculated value merely because it appears more confident.

Architecture decision record

Record the context, decision, alternatives, consequences and review trigger. Example: choose SQLite for the local prototype because one process and a small dataset meet the teaching need. PostgreSQL is an alternative if concurrent writers or managed access become requirements. Revisit the decision if measured contention or deployment requirements exceed the prototype assumptions.

A decision record is not a defence of the first idea forever. It preserves the reasoning so a later engineer can determine which assumption changed.

3 Interfaces and contracts

An interface describes available operations, input and output structure, error behaviour and relevant side effects. For a reading record, specify device_id, timestamp, power_W, interval_min and source identifier. Use a consistent timestamp convention, such as UTC, and distinguish the start of an interval from its end.

For an HTTP service, an API is an externally accessible interface using requests and responses. GET commonly retrieves a resource; POST commonly submits data or initiates an operation. State codes and response bodies should distinguish invalid input, unauthorised access, missing data and temporary service failure. Do not return a successful status with a fabricated result after an internal exception.

Idempotency means repeating the same intended operation has the same effect as doing it once. A retry of a read is often straightforward; a retry that inserts a report or sends a notification may duplicate effects. Use a request identifier and explicit deduplication policy for repeatable writes. Model-generated identifiers should not determine a user's access rights.

For a local project, a Python function or command-line interface can implement the same conceptual boundary without a network server. Build the HTTP layer only if it answers a project requirement.

4 Relational data and provenance

A relational database stores rows in tables with declared fields and relationships. A primary key identifies a row. A foreign key links related records. Normalisation reduces unintended duplication and update anomalies by separating facts with different dependencies.

Use a devices table for stable device metadata and a readings table for observations. Repeating the device's model and location in every reading makes updates inconsistent. Keep raw observations and derived results distinguishable so that a recalculated result does not overwrite the evidence that produced it.

Table	Example fields	Key idea
devices	device_id, description	One record per simulated device
readings	device_id, timestamp_utc, power_W, interval_min	One observation per device and interval
documents	document_id, revision, approved_at	Versioned reference material
evaluations	run_id, code_version, model_id, data_revision	Traceable experimental result

Provenance records where data came from and how it was transformed. For a sensor dataset, capture the generator or source, units, sampling meaning, timestamps, missing-data policy and version. For a retrieved manual, capture document identity, revision and passage identifier. A citation to an unknown revision may not support the current equipment configuration.

SQL example

```
rows = connection.execute(
    "SELECT power_W FROM readings WHERE device_id = ? "
    "ORDER BY timestamp_utc",
    (device_id,),
).fetchall()
```

The placeholder binds a value without building SQL syntax from user text. The trailing comma makes a one-element tuple. Parameter binding protects values, not arbitrary table names or query fragments; keep those under application control. Python's SQLite documentation provides the authoritative interface details. [R1]

Indexes can accelerate selected lookups at the cost of storage and write work. An index on device_id and timestamp_utc may support per-device time-range queries. Measure with the actual query and data scale rather than adding an index to every field.

5 Time series and leakage

Telemetry records have time relationships. Randomly splitting overlapping windows from the same recording across development and test sets can make evaluation unrealistically easy. Adjacent windows may share nearly all samples. Split by device, session or time period according to the intended generalisation question.

Distinguish duplicate timestamps, out-of-order arrival and missing intervals. Sorting can fix arrival order but cannot recover missing data. A complete energy report requires coverage information. If interval durations vary, use $\sum(P_i \times d_i / 60)$ rather than a single common duration.

For model evaluation, preprocessing and thresholds chosen using the test data leak information. Fit or select those decisions on training or development data, then freeze them before final evaluation. A synthetic dataset can test software mechanics, but it does not establish field performance on real equipment.

6 Retrieval before generation

Retrieval finds potentially relevant information in a collection. A simple keyword search may be sufficient for a small set of approved manuals. Embedding retrieval represents text as numeric vectors and ranks candidates using a similarity measure. Similarity is not truth, authorisation or evidence that a passage contains the answer.

Retrieval-augmented generation, or RAG, supplies retrieved material as context to a model when it answers. It does not automatically train the model on that material. Fine-tuning changes model parameters through a training process; prompt context and retrieval normally do not. Choose among lookup, RAG and training based on a measured need rather than their novelty.

A retrieval pipeline ingests approved documents, records revisions, divides them into passages, builds an index, retrieves candidates and returns passage identifiers with text. Chunk boundaries matter: separating a warning from the condition it qualifies can produce misleading context. Preserve units, headings and necessary surrounding information.

The supplied `retrieval_demo.py` uses deliberately simple lexical matching over invented manual passages. Its purpose is to make ranking and missed matches visible. It is neither an embedding model nor a complete production RAG system.

Worked retrieval example

The query is "motor overheating inspection". A passage mentioning motor temperature and blocked ventilation may be relevant. Another passage about network timeout contains the word inspection but addresses a different problem. A lexical ranker may miss a passage that uses "thermal" instead of "overheating". An embedding ranker may retrieve semantically related but procedurally incorrect content.

Before generation, check whether the required evidence was retrieved. If it was absent, changing the answer prompt cannot supply that missing source reliably. A fallback should say that the approved material does not support a conclusion.

7 Evaluate retrieval and answers separately

For a query with R relevant passages, recall at k is the number of relevant passages in the top k divided by R . Precision at k is the relevant count divided by k , assuming k results were returned. If two relevant passages exist and one appears among three returned passages, recall is $1/2$ and precision is $1/3$.

These metrics require relevance labels and a defined corpus. If the denominator is unknown, do not claim a measured recall. Average results across query types, including no-answer questions, revision-sensitive questions and misleading near-matches.

Answer evaluation asks different questions: are claims supported by the cited passages, does the response answer the question, are units preserved, and does it abstain when evidence is missing? A real passage identifier does not establish that the passage supports the statement. Assess citation correctness, not merely citation presence.

Use deterministic checks for response structure and numerical constraints. Use a human-reviewed rubric for meaning. A model judge can assist with open-ended assessment, but calibrate it against human labels and inspect disagreements. Do not let fluent wording compensate for a false technical claim.

8 The replaceable AI adapter

An adapter translates the application contract into a provider-specific request and maps the response back. Keep provider identifiers, authentication, SDK details and network errors within that boundary. Tests can substitute a deterministic stub to exercise failures without incurring live cost.

The adapter's contract should include a deadline, input size limit, permitted output schema, selected model identifier, usage accounting and fallback policy. Store credentials outside source code in an approved secret mechanism. Record the code, prompt, model and document versions used in an evaluation.

The course example accepts an injected provider callable. It validates the result and falls back when the provider times out or returns invalid data. The stub has predetermined responses. Running it tests application handling, not a language model's reasoning, network timeout enforcement or a provider's availability.

Live integration preparation

For a live demonstration, select one approved provider, consult its current SDK documentation, create a restricted teaching credential, set an explicit request timeout and budget, and map its response into the course schema. Run a harmless prompt using only the invented manual text. Record the actual model identifier, SDK version, latency, reported usage and raw response with secrets removed. Then replay that response through the validator.

Do not claim that an application enforces a network deadline merely because it catches `TimeoutError`. The HTTP client or SDK must actually apply a timeout. An optional extension implements the live adapter and verifies timeout behaviour with a controlled test endpoint or supported client mock.

9 Structured output is a syntax contract

Ask for a compact structure such as status, summary and source_ids. Check required fields, allowed status values, text length and whether source identifiers belong to the authorised retrieved set. Reject unknown fields if the application contract requires an exact schema. Check size before parsing when processing arbitrary bytes.

```
{
  "status": "advice",
  "summary": "Inspect the simulated ventilation path.",
  "source_ids": ["manual_motor_v1_p2"]
}
```

The object is well-formed JSON. Its factual correctness still depends on the source and question. An allowed source ID can accompany a false statement. Validation must not turn structurally valid model output into a physical command.

A refusal or unavailable response is a normal case to design for. If the provider cannot return an acceptable answer, preserve the verified energy result and present advice as unavailable. Never replace the numerical calculation with a guessed value from an explanation.

10 Tools agents and MCP

Tool calling lets a model request a named application operation with structured arguments. The application decides whether the request is authorised and valid, executes the operation if permitted, and returns a result. A tool request is proposed action, not permission to act.

The MCP specification dated 28 July 2026 describes an open connection protocol with hosts, clients and servers, using JSON-RPC messages. It distinguishes resources, prompts and tools. [R2] Treat this as a versioned interface example; client and server compatibility must be checked when implementing it. Do not mix session assumptions from older tutorials with a different protocol revision.

A course tool might read one approved manual passage by ID. It should not accept an arbitrary filesystem path or a shell command. Enforce the allowed ID set in ordinary code and record the access. An instruction asking the model to behave safely is not equivalent to an enforced tool boundary.

The core lab uses a local function with the same conceptual boundary. A full MCP server is an extension after students can explain the underlying operation and authorisation. The protocol itself does not establish factual accuracy, safe permissions or data ownership.

11 Reliability and model changes

Define a limited retry policy for transient failures, with backoff and a total deadline. Do not retry invalid user data indefinitely. A circuit breaker temporarily stops repeated calls to a failing dependency so the rest of the application can recover. Its state and reset policy should be explicit and tested.

Cache keys must include factors that affect the answer, such as document revision, query, permissions and relevant model or prompt version. A cache shared across users must not leak one user's authorised content to another. If the required evidence changes, old answers may need invalidation.

A model upgrade is a software change. Run the same frozen evaluation cases, compare failures and resource use, inspect important regressions and retain a rollback path. A new model name is not sufficient evidence that the application improved.

Weekly laboratories

Week 8 lab Architecture and milestone 1

- Minutes 0 to 15: state two measurable quality scenarios and draw the component boundaries.
- Minutes 15 to 35: write one architecture decision record comparing two alternatives.
- Minutes 35 to 50: trace valid input and a failed advice call through the design.
- Minutes 50 to 60: submit milestone 1 with the deterministic baseline, tests and data contract.

The core deliverable is an understandable design and runnable baseline, not several deployed services.

Week 9 lab Query and retrieve

- Minutes 0 to 15: run the supplied SQLite example and inspect records and units.
- Minutes 15 to 30: query one device with a bound parameter and predict its total.

- Minutes 30 to 45: run three lexical queries and identify a missed or irrelevant passage.
- Minutes 45 to 60: calculate one retrieval metric from supplied relevance labels and record corpus limitations.

Extension: compare a different retrieval method using the same labelled queries.

Week 10 lab Adapter and failure handling

- Minutes 0 to 10: inspect the response schema and authorised source IDs.
- Minutes 10 to 30: run valid, malformed, unsupported-source and timeout stubs.
- Minutes 30 to 45: inspect why valid syntax does not prove a supported conclusion.
- Minutes 45 to 55: observe an approved live call or replay its recorded response, labelling which occurred.
- Minutes 55 to 60: submit the failure table and model or stub version record.

Installation and provider account preparation occur before the lab. The optional live adapter belongs in independent project work if setup would otherwise consume the teaching hour.

Practice questions

Question 1 Architecture

Why is a modular monolith a reasonable starting point for this project?

Model answer. It preserves testable boundaries while avoiding unnecessary deployment and network coordination. It is justified by a small team and local workload. Reconsider it if measured scaling, ownership or isolation requirements demand independent services.

Question 2 Quality requirement

Rewrite "the advice service must be reliable" as a testable scenario.

Model answer. For example, when the advice provider times out on a valid batch, the application returns the independently calculated energy and marks advice unavailable within the specified total deadline. State the deadline, workload and environment before measuring it.

Question 3 Interval meaning

Two interval averages are 60 W for 10 minutes and 120 W for 20 minutes. Calculate energy.

Model answer. Energy is $60 \times 10 / 60 + 120 \times 20 / 60 = 10 + 40 = 50$ Wh. A common-duration formula is inappropriate because durations differ.

Question 4 Parameter binding

Does a SQL value placeholder safely let a user choose any table name?

Model answer. No. Placeholders bind values, not arbitrary SQL identifiers or syntax. Keep table and column choices under application control, for example through an explicit allowlist.

Question 5 Provenance

Name four records needed to reproduce a manual-grounded advice evaluation.

Model answer. Record the query set and labels, document revision and passage IDs, code and prompt versions, and provider model identifier or replayed response set. Also record configuration and raw outputs where permitted. These identify what was actually evaluated.

Question 6 Retrieval metrics

There are three relevant passages. Two occur in the top five results. Calculate precision at five and recall at five.

Model answer. Precision is $2/5 = 0.4$. Recall is $2/3$, approximately 0.667. This assumes the relevance set is known and five results were returned.

Question 7 RAG and training

Does adding a manual passage to a prompt ordinarily retrain the model?

Model answer. No. It supplies inference-time context. Fine-tuning changes parameters through training. RAG may improve access to source information but still requires retrieval and answer evaluation.

Question 8 Structured output

A response passes its JSON schema and cites a permitted passage. Is it necessarily correct?

Model answer. No. The statement may misinterpret the passage or fail to answer the query. Schema validation checks structure and allowed values. Grounding requires checking that the source supports each material claim.

Question 9 Timeout handling

Why does catching `TimeoutError` not prove that a network request has a bounded duration?

Model answer. An error handler only reacts if an error is raised. The network client must enforce a timeout, and the complete workflow needs a total deadline that includes retries and other steps.

Question 10 Tool authority

A model requests a tool call for an unapproved document ID. What should the application do?

Model answer. Reject the request at the tool boundary using ordinary access-control logic, record the denied attempt as appropriate and return a controlled error. The model's request does not grant access.

Question 11 Model upgrade

A new model is cheaper and has a higher public benchmark score. Is that sufficient reason to replace the current adapter configuration?

Model answer. No. Run the application's frozen evaluation set, compare relevant failure cases, latency, cost and compatibility, and retain rollback. Public benchmarks may not represent the application's data or constraints.

Question 12 Time-series leakage

Why can randomly splitting overlapping sensor windows exaggerate a model's apparent generalisation?

Model answer. Development and test windows may share samples, device-specific patterns or neighbouring events. Split by the unit of intended generalisation, such as device or session, and avoid tuning preprocessing on the final test set.

Revision checklist

Draw the deterministic and AI paths separately. Defend one architecture decision. Explain a parameterised query. Calculate energy for unequal intervals. Distinguish recall from groundedness. Explain the limits of a stub and a schema validator. State the enforcement point for a tool permission and the evidence needed before a model upgrade.

Sources and further reading

- [R1] Python Software Foundation. sqlite3 documentation.
<https://docs.python.org/3/library/sqlite3.html>
- [R2] Model Context Protocol. Specification dated 28 July 2026.
<https://modelcontextprotocol.io/specification/2026-07-28>
- Lewis and colleagues. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks.
<https://arxiv.org/abs/2005.11401>
- Anthropic. Demystifying evals for AI agents.
<https://www.anthropic.com/engineering/demystifying-evals-for-ai-agents>
- Further reading: Kleppmann, Designing Data-Intensive Applications, selected chapters on data models, replication and reliability; Sommerville, Software Engineering, architecture chapters.

References checked 15 September 2026. Schemas, manual passages and service scenarios are classroom examples. Protocol-specific implementation details should be checked against the selected revision before a live lab.